

AWS - Master Notes

A Complete Reference for Interview Preparation, Production Work, Real-World Projects, and System Understanding.

Java Full Stack Master Course

Table of Contents

AWS — MASTER NOTES	6
A Complete Reference Book for Interviews, Production Work & System Understanding	6
Written for someone who has never used a cloud platform before — includes account setup, service usage models, and step-by-step console instructions	6

INTRODUCTION: WHAT IS AWS, AND WHAT ARE THE CLOUD SERVICE MODELS?	6
1. What is AWS?	6
2. Why Cloud Computing Exists — The Exact Problem It Solves	6
3. The Three Cloud Service Models — A Critical, Frequently-Tested Distinction	6
4. FULL STEPS: Setting Up Your First AWS Account (2026)	7
STEP 1 — Sign Up	7
STEP 2 — CRITICAL FIRST SECURITY STEP: Never Use the Root User Day-to-Day	7
STEP 3 — Set a Billing Alert (Avoid Surprise Charges)	7
5. Real Life Analogy	7
6. Frequently Asked Interview Questions	8

TOPIC 1: IAM (IDENTITY AND ACCESS MANAGEMENT)	8
1. Introduction	8
2. Key IAM Concepts	9
3. FULL STEPS: Creating Your First IAM User (Instead of Using Root)	9
4. Example IAM Policy (JSON)	9
5. Real Life Analogy	9
6. Frequently Asked Interview Questions	10

TOPIC 2: EC2 (ELASTIC COMPUTE CLOUD)	10
1. Introduction	10
2. FULL STEP-BY-STEP: Launching an EC2 Instance and Deploying Your Spring Boot App	10
STEP 1 — Launch an Instance	10
STEP 2 — Connect to Your Instance	11
STEP 3 — Install Java and Deploy Your App	11

STEP 4 — Keep It Running After You Disconnect	11
3. Instance Types — Choosing the Right Size	11
4. Real Life Analogy	11
5. Edge Cases	12
6. Common Mistakes	12
7. Frequently Asked Interview Questions	12

TOPIC 3: S3 (SIMPLE STORAGE SERVICE) 12

1. Introduction	13
2. FULL STEP-BY-STEP: Creating a Bucket and Uploading a File	13
STEP 1 — Create a Bucket	13
STEP 2 — Upload a File (Console)	13
STEP 3 — Upload/Access via the AWS SDK (Java, Tying Back to the File Upload Topic in Spring Boot)	13
3. S3 Storage Classes (Cost Optimization)	14
4. Real Life Analogy	14
5. Edge Cases	14
6. Common Mistakes	14
7. Frequently Asked Interview Questions	14

TOPIC 4: RDS (RELATIONAL DATABASE SERVICE) 15

1. Introduction	15
2. FULL STEP-BY-STEP: Creating a Managed PostgreSQL Database	15
STEP 1 — Create the Database	15
STEP 2 — Connect Your Spring Boot Application to It	15
3. Why RDS Instead of Self-Managing PostgreSQL on an EC2 Instance	16
4. Real Life Analogy	16
5. Frequently Asked Interview Questions	16

TOPIC 5: VPC (VIRTUAL PRIVATE CLOUD) — BASICS 16

1. Introduction	16
2. Key Concepts	17
3. A Typical, Secure Architecture	17
4. Real Life Analogy	17
5. Frequently Asked Interview Questions	17

TOPIC 6: LOAD BALANCER	18
1. Introduction	18
2. FULL STEPS: Setting Up an Application Load Balancer (ALB)	18
3. How Health Checks Work	18
4. Real Life Analogy	18
5. Frequently Asked Interview Questions	18
TOPIC 7: AUTO SCALING — BASICS	19
1. Introduction	19
2. Key Concepts	19
3. Code Example — A Scaling Policy Concept	19
4. Real Life Analogy	19
5. Frequently Asked Interview Questions	20
TOPIC 8: CLOUDWATCH	20
1. Introduction	20
2. Key Concepts	20
3. Code Example — Sending Custom Application Metrics	21
4. Setting Up an Alarm	21
5. Real Life Analogy	21
6. Frequently Asked Interview Questions	21
TOPIC 9: ROUTE 53	21
1. Introduction	22
2. FULL STEPS: Pointing a Domain to Your Load Balancer	22
3. Real Life Analogy	22
4. Frequently Asked Interview Questions	22
TOPIC 10: ECS (ELASTIC CONTAINER SERVICE) — BASICS	22
1. Introduction	23
2. FULL STEP-BY-STEP: Deploying a Containerized Spring Boot App to ECS (Fargate)	23
STEP 1 — Push Your Image to Amazon ECR (Elastic Container Registry)	23

STEP 2 — Create an ECS Cluster	23
STEP 3 — Define a Task Definition	23
STEP 4 — Create a Service (Runs and Maintains Your Desired Number of Containers)	23
3. Fargate vs EC2 Launch Type — A Key Choice	24
4. Real Life Analogy	24
5. Frequently Asked Interview Questions	24

TOPIC 11: LAMBDA — BASICS **24**

1. Introduction	24
2. FULL STEP-BY-STEP: Creating a Simple Lambda Function	25
STEP 1 — Write the Function (Java Example)	25
STEP 2 — Package and Upload	25
STEP 3 — Trigger It	25
3. When Lambda Makes Sense vs EC2/ECS	25
4. Real Life Analogy	25
5. Edge Cases	26
6. Frequently Asked Interview Questions	26
7. Revision Notes (Entire AWS Section Consolidated)	27
8. Homework (Covers Entire AWS Section)	27
FINAL SUMMARY — AWS COMPLETE	27

AWS — MASTER NOTES

A Complete Reference Book for Interviews, Production Work & System Understanding

Written for someone who has never used a cloud platform before — includes account setup, service usage models, and step-by-step console instructions

INTRODUCTION: WHAT IS AWS, AND WHAT ARE THE CLOUD SERVICE MODELS?

1. What is AWS?

AWS (Amazon Web Services) is a **cloud computing platform** — instead of buying, physically installing, and maintaining your OWN servers/hardware, you **RENT** computing resources (servers, storage, databases, networking) from Amazon's data centers, paying only for what you actually use, and scaling up/down on demand within minutes.

2. Why Cloud Computing Exists — The Exact Problem It Solves

BEFORE THE CLOUD (traditional "on-premises" hosting):

- Buy PHYSICAL servers upfront (expensive capital cost, weeks to procure/ship)
- Install and maintain them YOURSELF (power, cooling, networking, hardware failures)
- GUESS your future capacity needs - buy too much (wasted money) or too little (can't handle a traffic spike, e.g., a sudden viral product launch)
- Scaling UP takes WEEKS (order more hardware, install it, configure it)

WITH AWS:

- RENT exactly the capacity you need, TODAY, provisioned in MINUTES
- PAY-AS-YOU-GO - only pay for what you actually use, scale down when demand drops
- Amazon handles the PHYSICAL data center, hardware, power, and networking
- Scale from 1 server to 10,000 servers in minutes, automatically (Auto Scaling, Topic 7)

3. The Three Cloud Service Models — A Critical, Frequently-Tested Distinction

IaaS (Infrastructure as a Service) -- you get RAW computing resources
Example: EC2 (Topic 2) - a virtual server; YOU install the OS, runtime, application, everything - MAXIMUM control, MAXIMUM responsibility

PaaS (Platform as a Service) -- you get a MANAGED PLATFORM to run your app
Example: AWS Elastic Beanstalk, AWS Lambda (Topic 11) - Amazon manages the underlying servers/OS/scaling; YOU just provide your CODE

SaaS (Software as a Service) -- you just USE a complete, ready-made application
Example: Gmail, Salesforce - you don't manage ANY infrastructure at all, just use the finished software directly

Model	You Manage	Amazon Manages	Analogy
IaaS	OS, runtime, app, data, scaling logic	Physical hardware, data center, virtualization	Renting an EMPTY apartment — you bring your own furniture, cook your own food
PaaS	Just your application code	OS, runtime, servers, scaling	A serviced apartment — furnished, cleaning included, you just live there
SaaS	Nothing (just your data/usage)	Literally everything	A hotel room — fully serviced, you just check in and use it

This distinction is genuinely important: EC2 (Topic 2) is IaaS (you manage the OS and everything on it); Lambda (Topic 11) is closer to PaaS/serverless (Amazon manages the server entirely — you just supply code); most of the specific services covered in this section (RDS, S3) sit at VARIOUS points on this spectrum, each trading control for reduced operational burden.

4. FULL STEPS: Setting Up Your First AWS Account (2026)

STEP 1 — Sign Up

- 1 Go to <https://aws.amazon.com/> → click "Create an AWS Account".
- 2 Provide an email address, a password, and an AWS account name.
- 3 Provide contact information (personal or business).
- 4 Provide a PAYMENT METHOD (a credit/debit card) — required even for Free Tier usage, as a verification step; AWS will NOT charge you for services that stay within Free Tier limits.
- 5 Verify your identity via a phone call/SMS code.
- 6 Choose a support plan (the **Free / Basic plan** is sufficient for learning).

STEP 2 — CRITICAL FIRST SECURITY STEP: Never Use the Root User Day-to-Day

The account you just created is the **root user** — it has UNRESTRICTED access to EVERYTHING, including billing. **Immediately create a separate IAM user (Topic 1) for daily work**, and enable MFA (Multi-Factor Authentication) on the root account, then avoid logging in as root except for rare account-level tasks.

STEP 3 — Set a Billing Alert (Avoid Surprise Charges)

- 1 Go to the **Billing and Cost Management** console.
- 2 Navigate to **Budgets** → **Create a budget**.
- 3 Set a monthly budget threshold (e.g., \$5) and configure an email ALERT if you approach or exceed it — a genuinely important safety net while learning, since it's easy to accidentally leave a resource running.

5. Real Life Analogy

- **Restaurant:** AWS is like a MASSIVE, shared industrial kitchen complex that ANY restaurant business can rent space/equipment from — instead of every single restaurant chain needing to build and personally maintain its OWN power plant, cold storage facility, and equipment manufacturing line, they simply rent EXACTLY the kitchen capacity they need, scaling up during a busy holiday season and back down afterward, paying only for what they actually use.

6. Frequently Asked Interview Questions

- 1 **What is AWS?** A cloud computing platform providing on-demand, pay-as-you-go access to computing resources (servers, storage, databases, networking) without needing to own/maintain physical hardware.
 - 2 **What problem does cloud computing solve compared to traditional on-premises hosting?** Eliminates large upfront hardware costs, slow procurement/scaling timelines, and the burden of physically maintaining infrastructure — resources are provisioned in minutes and paid for only as used.
 - 3 **What are the three cloud service models, and give an AWS example of each?** IaaS (EC2 — raw virtual servers you fully manage), PaaS (Lambda/Elastic Beanstalk — Amazon manages the runtime, you provide code), SaaS (a fully finished application you just use, not built on raw AWS infrastructure yourself).
 - 4 **Why should you avoid using the AWS root user for daily work?** It has unrestricted access to everything including billing — a compromised root account is catastrophic; a separate, appropriately-scoped IAM user should be used instead for routine work.
-
-

TOPIC 1: IAM (IDENTITY AND ACCESS MANAGEMENT)

1. Introduction

What is it? IAM controls WHO (users, applications, services) can access WHAT AWS resources, and WHAT ACTIONS they're allowed to perform — the security foundation underlying EVERY other AWS service, directly analogous to Spring Security's Authentication/Authorization (RBAC) concepts, just applied to your CLOUD infrastructure itself rather than your application's own users.

2. Key IAM Concepts

Concept	Meaning
User	An individual identity (a person, or an application) with its own credentials
Group	A collection of users sharing the SAME set of permissions (exactly like a "role" in RBAC)
Role	A set of permissions ASSUMED TEMPORARILY by a user, application, or AWS SERVICE itself (e.g., letting an EC2 instance access an S3 bucket without embedding permanent credentials)
Policy	A JSON document defining EXACTLY which actions are allowed/denied on which resources

3. FULL STEPS: Creating Your First IAM User (Instead of Using Root)

- 1 In the AWS Console, navigate to **IAM** → **Users** → **Create user**.
- 2 Give it a name (e.g., `dev-admin`).
- 3 Choose **"Provide user access to the AWS Management Console"** if you want console login access.
- 4 Attach a permissions policy — for learning, `AdministratorAccess` is acceptable (grants broad access), but in a REAL company, you'd attach a much more NARROWLY-SCOPED policy (principle of least privilege — the SAME concept from Spring Security's Password Encoding/RBAC topics).
- 5 Complete creation — AWS shows you a sign-in URL specific to your account; use THIS (not the root login) for daily work.
- 6 Enable MFA on this user too, for genuine production-grade security.

4. Example IAM Policy (JSON)

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": ["s3:GetObject", "s3:PutObject"],
      "Resource": "arn:aws:s3:::my-company-bucket/*"
    }
  ]
}
```

This policy allows ONLY reading/writing objects in ONE SPECIFIC S3 bucket — NOT deleting buckets, NOT accessing other services, NOT touching billing — a deliberately narrow, "least privilege" grant.

5. Real Life Analogy

- **Bank:** IAM is like the bank's internal staff ID badge system — different badges (Users/Roles) are issued DIFFERENT levels of building/vault access (Policies), and a "Teller" badge (Group) automatically grants the SAME standard permissions to every teller, without the bank needing to individually configure access for each new hire from scratch.

6. Frequently Asked Interview Questions

- 1 **What is IAM?** AWS's identity and access management service, controlling who can access which AWS resources and what actions they can perform.
 - 2 **What's the difference between an IAM User, Group, and Role?** A User is an individual identity; a Group is a shared permission set assigned to multiple users; a Role is a temporarily-assumed permission set, often used by AWS services themselves (like an EC2 instance accessing S3).
 - 3 **What does "least privilege" mean in the context of IAM policies?** Granting only the MINIMUM permissions actually needed for a task, rather than broad, unrestricted access — reducing the damage if credentials are ever compromised.
-

TOPIC 2: EC2 (ELASTIC COMPUTE CLOUD)

1. Introduction

What is it? EC2 provides RESIZABLE VIRTUAL SERVERS ("instances") in the cloud — the core IaaS building block of AWS, letting you run ANYTHING you could run on a physical server (including your Spring Boot application, Docker containers, or a database) without owning physical hardware.

2. FULL STEP-BY-STEP: Launching an EC2 Instance and Deploying Your Spring Boot App

STEP 1 — Launch an Instance

- 1 In the AWS Console, go to **EC2** → **Launch Instance**.
- 2 Name it (e.g., `employee-api-server`).
- 3 Choose an **Amazon Machine Image (AMI)** — e.g., **Amazon Linux 2023** (a free-tier-eligible Linux distribution).
- 4 Choose an **Instance Type** — `t2.micro` or `t3.micro` (Free Tier eligible, suitable for learning/small workloads).
- 5 Create (or select an existing) **Key Pair** — download the `.pem` private key file; THIS is how you'll securely SSH into the instance (KEEP IT SAFE — it cannot be re-downloaded).
- 6 Configure the **Security Group** (a virtual firewall, Topic 5's networking concepts) — allow inbound traffic on port 22 (SSH) from your own IP, and port 8080 (your Spring Boot app) from anywhere (`0.0.0.0/0`), for now.
- 7 Click **Launch Instance**.

STEP 2 — Connect to Your Instance

```
chmod 400 my-key.pem # restrict key file permissions (required by SSH)
ssh -i my-key.pem ec2-user@<your-instance-public-ip> # connect via SSH
```

STEP 3 — Install Java and Deploy Your App

```
sudo yum install -y java-17-amazon-corretto # install Java 17 (Amazon's own OpenJDK distribution)

# Copy your built jar FROM your local machine TO the EC2 instance (run this on YOUR laptop, not the
server):
scp -i my-key.pem target/employee-api-1.0.0.jar ec2-user@<instance-ip>:/home/ec2-user/

# Back on the EC2 instance:
java -jar employee-api-1.0.0.jar
```

Your Spring Boot application is now running on a REAL cloud server, reachable at
<http://<instance-public-ip>:8080>.

STEP 4 — Keep It Running After You Disconnect

```
nohup java -jar employee-api-1.0.0.jar > app.log 2>&1 & # runs in the background, survives SSH
disconnect
# (in real production, you'd use a proper process manager or run it as a Docker container instead -
see Topic 10, ECS)
```

3. Instance Types — Choosing the Right Size

Family	Best For
t (e.g., t3.micro)	General-purpose, burstable — small apps, dev/test, low sustained load
m (e.g., m5.large)	Balanced general-purpose — typical production application servers
c (e.g., c5.large)	Compute-optimized — CPU-intensive workloads
r (e.g., r5.large)	Memory-optimized — large caches, in-memory databases

4. Real Life Analogy

- **Restaurant:** An EC2 instance is like renting a FULLY EQUIPPED but otherwise EMPTY kitchen space — Amazon provides the physical building/power/plumbing (the "instance"), but YOU install your own specific equipment, recipes, and staff (your OS configuration, application, runtime) — full control, full responsibility.

5. Edge Cases

Case	Result
Losing your <code>.pem</code> key file	Cannot SSH into that instance anymore through the normal method — recovery requires more advanced AWS-specific procedures (detaching the volume, mounting it to another instance)
Leaving a Security Group open to <code>0.0.0.0/0</code> on SSH (port 22)	A serious security risk — restrict SSH access to your OWN IP address specifically in any real deployment
Stopping (not terminating) an instance	You stop paying for COMPUTE time, but still pay for attached STORAGE (EBS volumes) until the instance/volume is fully terminated/deleted

6. Common Mistakes

- Leaving Security Groups wide open (`0.0.0.0/0`) on sensitive ports like SSH — should be restricted to known IP ranges.
- Forgetting instances still ACCRUE storage charges even when STOPPED (only fully TERMINATING avoids ongoing storage costs).
- Running a production application directly via `java -jar` with no process supervision — should use a proper service manager, or better, containerize it (Docker/ECS, Topic 10).

7. Frequently Asked Interview Questions

- 1 **What is EC2?** AWS's Infrastructure-as-a-Service offering — resizable virtual servers ("instances") you fully configure and manage.
- 2 **What is a Security Group?** A virtual firewall controlling inbound/outbound traffic allowed to/from an EC2 instance.
- 3 **What's the difference between STOPPING and TERMINATING an EC2 instance regarding cost?** Stopping halts compute charges but STILL incurs storage charges for attached volumes; terminating removes the instance (and typically its storage) entirely, stopping all associated charges.
- 4 **Why is `t3.micro` a common choice for learning/small workloads?** It's Free-Tier eligible and provides "burstable" performance suitable for low, intermittent load — not for sustained heavy production traffic.

TOPIC 3: S3 (SIMPLE STORAGE SERVICE)

1. Introduction

What is it? S3 is AWS's OBJECT STORAGE service — designed to store and retrieve ANY amount of data (files, images, backups, static website assets) with extremely high durability, WITHOUT you managing any actual servers/disks yourself.

2. FULL STEP-BY-STEP: Creating a Bucket and Uploading a File

STEP 1 — Create a Bucket

- 1 Go to **S3** → **Create bucket**.
- 2 Give it a GLOBALLY UNIQUE name (e.g., `jobsradar-employee-uploads-2026`) — bucket names must be unique across ALL of AWS, not just your account.
- 3 Choose a region (pick one geographically close to your users for lower latency).
- 4 Leave **"Block all public access"** ENABLED (the default, and correct choice) unless you specifically intend to host public content.
- 5 Create the bucket.

STEP 2 — Upload a File (Console)

- 1 Open the bucket → **Upload** → select a file → **Upload**.

STEP 3 — Upload/Access via the AWS SDK (Java, Tying Back to the File Upload Topic in Spring Boot)

```
<dependency>
<groupId>software.amazon.awssdk</groupId>
<artifactId>s3</artifactId>
</dependency>

@Service
public class S3StorageService {
    private final S3Client s3Client = S3Client.builder().region(Region.US_EAST_1).build();
    private static final String BUCKET = "jobsradar-employee-uploads-2026";

    public void uploadFile(String key, MultipartFile file) throws IOException {
        s3Client.putObject(
            PutObjectRequest.builder().bucket(BUCKET).key(key).build(),
            RequestBody.fromInputStream(file.getInputStream(), file.getSize())
        );
    }

    public byte[] downloadFile(String key) {
        return s3Client.getObject(GetObjectRequest.builder().bucket(BUCKET).key(key).build()).readAllBytes();
    }
}
```

This directly REPLACES the local-disk file storage shown in Spring Boot's File Upload topic — instead of `Files.copy(..., localPath)`, files go to S3, which is DURABLE, SCALABLE, and accessible from ANY server (unlike local disk storage, which ties uploaded files to ONE specific EC2 instance — a real problem if you're running MULTIPLE instances behind a Load Balancer, Topic 6).

3. S3 Storage Classes (Cost Optimization)

Class	Use Case
S3 Standard	Frequently accessed data (the default)
S3 Infrequent Access (IA)	Data accessed rarely, but needed quickly when it IS accessed
S3 Glacier	Long-term archival — very cheap storage, but SLOW retrieval (minutes to hours)

4. Real Life Analogy

- **Library:** S3 is like an ENORMOUS, effectively limitless, professionally-managed off-site storage warehouse — you hand over any item (file) with a label (key), and can request it back at any time, without needing to personally build/maintain the physical warehouse, its shelving, fire suppression, or security yourself.

5. Edge Cases

Case	Result
Trying to create a bucket with a name ALREADY used by ANYONE else globally	Fails — bucket names must be globally unique across all AWS accounts, not just your own
Leaving a bucket PUBLICLY accessible unintentionally	A serious, common real-world security incident — always leave "Block all public access" ON unless a bucket is DELIBERATELY meant to serve public content

6. Common Mistakes

- Accidentally making an S3 bucket public, exposing sensitive files to anyone on the internet — a genuinely common, serious real-world incident.
- Storing user-uploaded files on a single EC2 instance's local disk instead of S3, breaking when running multiple instances behind a load balancer (each instance would only see files uploaded directly to IT).

7. Frequently Asked Interview Questions

- 1 **What is S3?** AWS's object storage service, designed for storing and retrieving any amount of data with high durability, without managing physical storage infrastructure yourself.
- 2 **Why should S3 buckets storing user-uploaded files be preferred over local EC2 disk storage in a multi-instance deployment?** Local disk storage ties uploaded files to ONE specific instance; S3 is accessible from ANY instance/service, working correctly regardless of which server handled the original upload or which server later needs to retrieve it.
- 3 **What's a common, serious real-world S3 security mistake?** Accidentally leaving a bucket containing sensitive data publicly accessible.
- 4 **What's S3 Glacier used for?** Long-term data archival — very low storage cost, but with slow (minutes-to-hours) retrieval times, appropriate for rarely-accessed backup/compliance data.

TOPIC 4: RDS (RELATIONAL DATABASE SERVICE)

1. Introduction

What is it? RDS is AWS's MANAGED relational database service — running PostgreSQL, MySQL, or other standard databases WITHOUT you needing to install, patch, back up, or manage the underlying server yourself, directly replacing the self-hosted PostgreSQL setup from the PostgreSQL section's local installation steps.

2. FULL STEP-BY-STEP: Creating a Managed PostgreSQL Database

STEP 1 — Create the Database

- 1 Go to **RDS** → **Create database**.
- 2 Choose **Standard create** and engine type **PostgreSQL**.
- 3 Choose a template — **Free tier** for learning.
- 4 Set a DB instance identifier (e.g., `company-db`), master username, and password.
- 5 Choose instance size (`db.t3.micro` for Free Tier).
- 6 Configure connectivity — for learning, allow "Public access: Yes" (in REAL production, this would typically be "No," with access restricted to your application servers only, via VPC configuration, Topic 5).
- 7 Create the database (provisioning takes several minutes).

STEP 2 — Connect Your Spring Boot Application to It

```
# application.properties - now pointing to RDS instead of local PostgreSQL
spring.datasource.url=jdbc:postgresql://company-db.abc123xyz.us-east-1.rds.amazonaws.com:5432/company_db
spring.datasource.username=postgres
spring.datasource.password=${DB_PASSWORD} # NEVER hardcode - inject via environment variable
```

Notice: this is the EXACT SAME `spring.datasource.*` configuration mechanism from the Spring Boot/PostgreSQL sections — RDS is simply a DIFFERENT, managed HOSTING location for the SAME PostgreSQL database engine you already know how to use.

3. Why RDS Instead of Self-Managing PostgreSQL on an EC2 Instance

Aspect	Self-Managed PostgreSQL on EC2	RDS (Managed)
OS/database patching	YOUR responsibility	Handled AUTOMATICALLY by AWS
Backups	YOU must configure/schedule (pg_dump, PostgreSQL section)	Automated daily backups + point-in-time recovery, built in
High availability (failover)	YOU must build it yourself	Available with one configuration option (Multi-AZ deployment)
Scaling	Manual	Can be resized with a few clicks

4. Real Life Analogy

- **Bank:** Self-managed PostgreSQL on EC2 is like a bank BUILDING AND MAINTAINING its own private vault facility — full control, but they're responsible for EVERY security update, structural repair, and disaster backup themselves. RDS is like RENTING vault space in a professionally-managed, third-party secure facility that handles all the maintenance/security/backup responsibilities FOR you — you just use the vault.

5. Frequently Asked Interview Questions

- 1 **What is RDS?** AWS's managed relational database service, running standard engines like PostgreSQL/MySQL without requiring you to manually install, patch, or back up the database server yourself.
- 2 **Name three operational responsibilities RDS takes off your hands compared to self-managed PostgreSQL on EC2.** Automated backups/point-in-time recovery, OS/database patching, and (optionally) built-in high-availability failover (Multi-AZ).
- 3 **Does connecting a Spring Boot application to RDS require different code than connecting to a local PostgreSQL instance?** No — only the connection URL/credentials change; the same `spring.datasource.*` JDBC configuration mechanism is used identically either way.

TOPIC 5: VPC (VIRTUAL PRIVATE CLOUD) — BASICS

1. Introduction

What is it? A VPC is your OWN ISOLATED, PRIVATE section of the AWS cloud — a virtual network where you control IP address ranges, subnets, and routing, letting you decide EXACTLY which resources are PUBLICLY reachable from the internet and which are kept PRIVATE, internal-only.

2. Key Concepts

Concept	Meaning
VPC	Your isolated virtual network within AWS
Subnet	A subdivision of your VPC's IP range, tied to a specific Availability Zone
Public Subnet	Has a route to the internet (via an Internet Gateway) — for internet-facing resources (a Load Balancer)
Private Subnet	NO direct route to the internet — for resources that should NEVER be directly reachable (a database)
Security Group	A firewall applied at the INSTANCE level (recap from Topic 2)

3. A Typical, Secure Architecture

```
VPC
+-----+
| PUBLIC SUBNET PRIVATE SUBNET |
| +-----+ +-----+ |
| | Load Balancer | -----> | EC2 (App Server) | |
| | (internet-facing) | +-----+ |
| +-----+ | |
| ^ v |
| | +-----+ |
+-----+-----+ RDS (Database) |-----+
| +-----+
INTERNET (users) (NO direct internet access - only reachable
from the App Server inside the SAME VPC)
```

This is EXACTLY why RDS Step 1's "Public access" setting matters — in a REAL production setup, the database sits in a PRIVATE subnet, reachable ONLY from your application servers within the same VPC, never directly from the public internet.

4. Real Life Analogy

- **Bank:** A VPC is like a bank's ENTIRE private office building — the PUBLIC subnet is like the customer-facing lobby (reachable from the street), while the PRIVATE subnet is like the internal vault and back offices (reachable ONLY from within the building itself, never directly from the street outside).

5. Frequently Asked Interview Questions

- 1 **What is a VPC?** Your own isolated, private virtual network within AWS, where you control IP ranges, subnets, and routing.
- 2 **What's the difference between a public and private subnet?** A public subnet has a route to the internet (via an Internet Gateway); a private subnet does not, keeping resources inside it unreachable directly from the internet.
- 3 **Why would a database typically live in a private subnet rather than a public one?** To prevent it from being directly reachable from the public internet at all — only application servers within the same VPC (private network) should be able to connect to it, significantly reducing the attack surface.

TOPIC 6: LOAD BALANCER

1. Introduction

What is it? A Load Balancer distributes incoming traffic ACROSS MULTIPLE server instances — essential once you have more than one EC2/container instance running your application, ensuring no single instance is overwhelmed, and automatically routing traffic AWAY from any UNHEALTHY instance.

2. FULL STEPS: Setting Up an Application Load Balancer (ALB)

- 1 Go to **EC2** → **Load Balancers** → **Create Load Balancer** → choose **Application Load Balancer**.
- 2 Name it, choose your VPC and PUBLIC subnets (across multiple Availability Zones for resilience).
- 3 Configure a **Security Group** allowing inbound HTTP/HTTPS traffic from the internet.
- 4 Create a **Target Group** — this is the pool of actual EC2 instances (or containers) the Load Balancer will route to; configure a **Health Check path** (e.g., `/actuator/health`, tying directly back to Spring Boot Actuator).
- 5 Register your EC2 instances (or, in ECS, this happens automatically, Topic 10) as targets.
- 6 Create the Load Balancer — it's now assigned a public DNS name that clients connect to, INSTEAD of any individual instance's own address.

3. How Health Checks Work

```
Load Balancer periodically calls: GET http://<instance>:8080/actuator/health
-> 200 OK -> instance stays IN the rotation, receives traffic
-> anything else / timeout -> instance is marked UNHEALTHY, REMOVED from
rotation automatically, until it starts passing health checks again
```

This ties DIRECTLY back to Docker's `HEALTHCHECK` concept and the Microservices section's resilience patterns — the SAME idea (automatically detect and route around unhealthy instances), applied here at the AWS infrastructure level.

4. Real Life Analogy

- **Restaurant:** A Load Balancer is like a restaurant chain's central RESERVATION/SEATING system that automatically directs incoming customers to WHICHEVER currently-open, healthy location has capacity — and instantly stops sending customers to any location that's temporarily closed/malfunctioning (failing its health check), without customers needing to know or care which specific location they end up at.

5. Frequently Asked Interview Questions

- 1 **What does a Load Balancer do?** Distributes incoming traffic across multiple server/container instances, and automatically routes traffic away from unhealthy instances.
- 2 **What is a health check, and how does the Load Balancer use it?** A periodic request (e.g., to /actuator/health) confirming an instance is functioning correctly; instances failing their health check are automatically removed from traffic rotation until they recover.
- 3 **Why is a Load Balancer necessary once you run more than one application instance?** Without it, clients would need to know and pick a specific instance's address themselves, with no automatic way to avoid overwhelmed or unhealthy instances.

TOPIC 7: AUTO SCALING — BASICS

1. Introduction

What is it? Auto Scaling AUTOMATICALLY adjusts the NUMBER of running instances based on actual demand — adding MORE instances during high traffic, and REMOVING instances during low traffic, WITHOUT manual intervention, directly enabling the "pay only for what you use" promise of cloud computing (Introduction topic).

2. Key Concepts

Concept	Meaning
Launch Template	Defines WHAT a new instance should look like (AMI, instance type, security group) when Auto Scaling creates one
Auto Scaling Group (ASG)	Manages a collection of instances, enforcing MINIMUM, DESIRED, and MAXIMUM instance counts
Scaling Policy	The RULE deciding WHEN to scale (e.g., "add an instance if average CPU exceeds 70% for 5 minutes")

3. Code Example — A Scaling Policy Concept

```
Auto Scaling Group: min=2, desired=2, max=10
```

```
Scaling Policy: "Target Tracking" on Average CPU Utilization = 50%
-> if average CPU across all instances RISES above 50%, ADD instances
-> if average CPU DROPS well below 50%, REMOVE instances
-> NEVER goes below 2 (min) or above 10 (max) instances
```

4. Real Life Analogy

- **Restaurant:** Auto Scaling is like a restaurant chain AUTOMATICALLY opening additional temporary service counters the MOMENT a line starts building up beyond a comfortable wait time, and AUTOMATICALLY closing those extra counters again once the rush subsides — all without a manager needing to manually notice the crowd and decide to open/close counters themselves.

5. Frequently Asked Interview Questions

- 1 **What does Auto Scaling do?** Automatically adjusts the number of running instances based on actual demand, adding instances under high load and removing them under low load.
 - 2 **What are the min/desired/max settings in an Auto Scaling Group?** The boundaries and target for the group's instance count — it will never scale below "min" or above "max," and actively tries to maintain "desired" (adjusted dynamically by scaling policies based on actual metrics).
 - 3 **How does Auto Scaling directly support the cloud's "pay only for what you use" value proposition?** By removing unneeded instances during low-demand periods automatically, you avoid paying for idle capacity you don't currently need, while still having capacity available immediately during demand spikes.
-

TOPIC 8: CLOUDWATCH

1. Introduction

What is it? CloudWatch is AWS's MONITORING and OBSERVABILITY service — collecting METRICS (CPU usage, request counts, error rates), LOGS, and enabling ALARMS (automated alerts/actions) across virtually every AWS service, directly analogous to the Distributed Logging/Tracing concepts from the Microservices section, but at the AWS INFRASTRUCTURE level.

2. Key Concepts

Concept	Meaning
Metrics	Numerical data points over time (CPU%, request count, error count)
Logs	Application/system log output, centrally collected (e.g., your Spring Boot app's logs, streamed from EC2/ECS)
Alarms	Automated triggers based on metric thresholds (e.g., "alert me if CPU > 90% for 5 minutes")
Dashboards	Visual displays combining multiple metrics/graphs

3. Code Example — Sending Custom Application Metrics

```
@Service
public class OrderMetricsService {
    private final CloudWatchClient cloudWatchClient = CloudWatchClient.create();

    public void recordOrderPlaced() {
        cloudWatchClient.putMetricData(PutMetricDataRequest.builder()
            .namespace("EmployeeApp")
            .metricData(MetricDatum.builder()
                .metricName("OrdersPlaced")
                .value(1.0)
                .unit(StandardUnit.COUNT)
                .build())
            .build());
    }
}
```

4. Setting Up an Alarm

- 1 Go to **CloudWatch** → **Alarms** → **Create alarm**.
- 2 Select a metric (e.g., an EC2 instance's `CPUUtilization`).
- 3 Set a threshold condition (e.g., "greater than 80% for 3 consecutive periods of 5 minutes").
- 4 Configure a notification action (e.g., send an email via SNS, or trigger an Auto Scaling action, Topic 7).

5. Real Life Analogy

- **Restaurant:** CloudWatch is like a restaurant chain's CENTRAL dashboard tracking every location's real-time metrics (customer wait times, kitchen temperatures, order volume) and AUTOMATICALLY alerting management the moment any metric crosses a concerning threshold — instead of relying on someone happening to notice a problem in person.

6. Frequently Asked Interview Questions

- 1 **What is CloudWatch?** AWS's monitoring and observability service, collecting metrics and logs and enabling automated alarms across AWS resources.
- 2 **What is a CloudWatch Alarm?** An automated trigger that fires when a monitored metric crosses a defined threshold, capable of sending notifications or triggering automated actions (like an Auto Scaling adjustment).
- 3 **How does CloudWatch relate to the Distributed Logging/Tracing concepts from the Microservices section?** It serves the SAME purpose (centralized observability) but specifically at the AWS infrastructure/service level, complementing application-level tools like ELK/Zipkin.

TOPIC 9: ROUTE 53

1. Introduction

What is it? Route 53 is AWS's DNS (Domain Name System) service — translating human-readable domain names (`www.mycompany.com`) into the actual IP addresses/AWS resources (a Load Balancer, an S3 bucket) that serve the request, and also handling domain REGISTRATION.

2. FULL STEPS: Pointing a Domain to Your Load Balancer

- 1 Go to **Route 53** → **Hosted zones** → create a hosted zone for your domain (or register a NEW domain directly through Route 53's domain registration feature).
- 2 Create a new RECORD:
 - Record type: **A** (or an **ALIAS** record specifically, for pointing directly at AWS resources like a Load Balancer).
 - Value: select your Application Load Balancer (Topic 6) from the dropdown.
- 3 Save — after DNS propagation (often near-instant for AWS-internal Alias records, but can take longer for standard DNS changes), `www.mycompany.com` now resolves directly to your Load Balancer.

3. Real Life Analogy

- **Bank:** Route 53 is like the bank's official PHONE DIRECTORY listing — customers dial a simple, memorable number ("Call MyBank") instead of needing to know the bank's actual internal technical extension number directly; the directory service translates the friendly name into the correct underlying destination.

4. Frequently Asked Interview Questions

- 1 **What is Route 53?** AWS's DNS service, translating domain names into the actual AWS resources/IP addresses that serve requests, and also supporting domain registration.
- 2 **What's the difference between an A record and an ALIAS record when pointing to an AWS Load Balancer?** An **ALIAS** record is an AWS-specific extension that can point directly at AWS resources (like a Load Balancer) by name, without needing a fixed IP address, since Load Balancer IPs can change; a plain A record requires a fixed IP.

TOPIC 10: ECS (ELASTIC CONTAINER SERVICE) — BASICS

1. Introduction

What is it? ECS is AWS's CONTAINER ORCHESTRATION service — running your DOCKER containers (from the Docker section) at scale in AWS, without you manually managing individual EC2 instances, handling the EXACT orchestration responsibilities (scaling, health-based routing, rolling deployments) previewed conceptually in the Docker and Microservices sections.

2. FULL STEP-BY-STEP: Deploying a Containerized Spring Boot App to ECS (Fargate)

STEP 1 — Push Your Image to Amazon ECR (Elastic Container Registry)

```
aws ecr create-repository --repository-name employee-api
docker tag employee-api:1.0.0 123456789.dkr.ecr.us-east-1.amazonaws.com/employee-api:1.0.0
docker push 123456789.dkr.ecr.us-east-1.amazonaws.com/employee-api:1.0.0
```

STEP 2 — Create an ECS Cluster

- 1 Go to **ECS** → **Clusters** → **Create cluster**.
- 2 Choose **AWS Fargate** — a SERVERLESS option meaning you never manage the underlying EC2 instances yourself at all (a PaaS-like experience layered on top of container orchestration).

STEP 3 — Define a Task Definition

```
{
  "family": "employee-api-task",
  "containerDefinitions": [
    {
      "name": "employee-api",
      "image": "123456789.dkr.ecr.us-east-1.amazonaws.com/employee-api:1.0.0",
      "portMappings": [{ "containerPort": 8080 }],
      "environment": [{ "name": "SPRING_PROFILES_ACTIVE", "value": "prod" }]
    }
  ],
  "requiresCompatibilities": ["FARGATE"],
  "cpu": "256",
  "memory": "512"
}
```

STEP 4 — Create a Service (Runs and Maintains Your Desired Number of Containers)

- 1 In your cluster, create a **Service** using the Task Definition above.
- 2 Set the desired COUNT of running containers (e.g., 3).
- 3 Attach it to your Application Load Balancer (Topic 6) — ECS AUTOMATICALLY registers/deregisters containers as targets, based on health checks.

3. Fargate vs EC2 Launch Type — A Key Choice

Aspect	EC2 Launch Type	Fargate (Serverless)
Who manages the underlying servers?	YOU (patch, size, scale the EC2 instances yourself)	AWS (fully managed — you never see/manage a server)
Control	More control over the underlying instance	Less control, but far less operational burden
Billing	Pay for the EC2 instances, regardless of container utilization	Pay per-container based on actual CPU/memory requested

4. Real Life Analogy

- **Restaurant:** ECS is like a restaurant chain's central OPERATIONS system that automatically opens/closes/replaces individual cooking STATIONS (containers) as needed across ALL its locations — Fargate specifically is like renting those stations FULLY STAFFED AND MAINTAINED by a third party, so the restaurant chain NEVER even needs to think about the physical equipment itself, just the actual RECIPES (containers) they want running.

5. Frequently Asked Interview Questions

- 1 **What is ECS?** AWS's container orchestration service, running Docker containers at scale, handling scaling/health-based routing/deployments.
- 2 **What is AWS Fargate?** A serverless launch type for ECS, where AWS manages the underlying compute infrastructure entirely — you never provision or manage EC2 instances yourself.
- 3 **What is Amazon ECR?** AWS's managed, private container image registry — the AWS-specific equivalent of the private registry concept covered in the Docker section.
- 4 **What's the main trade-off between the EC2 and Fargate launch types for ECS?** EC2 gives more control over the underlying instances (but requires managing them); Fargate removes that management burden entirely (serverless), at the cost of somewhat less low-level control.

TOPIC 11: LAMBDA — BASICS

1. Introduction

What is it? AWS Lambda is a SERVERLESS COMPUTE service — you write and upload a FUNCTION (a small piece of code), and AWS runs it ON DEMAND, in response to a TRIGGER (an HTTP request, a file uploaded to S3, a scheduled time), automatically scaling and CHARGING ONLY for the actual execution time — you never provision, manage, or pay for an idle server AT ALL.

2. FULL STEP-BY-STEP: Creating a Simple Lambda Function

STEP 1 — Write the Function (Java Example)

```
public class HelloLambdaHandler implements RequestHandler<Map<String, Object>, String> {  
    @Override  
    public String handleRequest(Map<String, Object> input, Context context) {  
        String name = (String) input.getOrDefault("name", "World");  
        return "Hello, " + name + "! (from AWS Lambda)";  
    }  
}
```

STEP 2 — Package and Upload

```
mvn clean package # produces a jar  
# Upload this jar via the Lambda console, or via the AWS CLI:  
aws lambda create-function --function-name hello-lambda \  
--runtime java17 --handler com.jobsradar.HelloLambdaHandler::handleRequest \  
--role arn:aws:iam::123456789:role/lambda-execution-role \  
--zip-file fileb://target/hello-lambda-1.0.0.jar
```

STEP 3 — Trigger It

- **Directly:** via the AWS Console's "Test" feature, or the AWS CLI (`aws lambda invoke`).
- **Via API Gateway:** expose it as an HTTP endpoint, callable from any client.
- **Via an S3 event:** automatically run WHENEVER a new file is uploaded to a bucket (e.g., automatically resizing an uploaded image).
- **Via a scheduled CloudWatch Event:** run on a cron-like schedule (conceptually similar to Spring's `@Scheduled`, but running SERVERLESS, with no application even needing to be continuously running at all).

3. When Lambda Makes Sense vs EC2/ECS

Scenario	Better Fit
A long-running, always-on Spring Boot API handling steady traffic	EC2/ECS
Infrequent, event-triggered tasks (resize an image on upload, run a nightly cleanup job, process a queue message occasionally)	Lambda
Highly SPIKY, unpredictable traffic where you don't want to pay for idle capacity AT ALL	Lambda
An application needing to maintain LONG-LIVED in-memory state or connections between requests	EC2/ECS (Lambda functions are stateless, short-lived, and can have unpredictable "cold start" latency)

4. Real Life Analogy

- **Restaurant:** Lambda is like hiring a specialist chef WHO ONLY GETS PAID AND SHOWS UP for the EXACT MINUTES they're actually preparing a specific, occasional specialty dish someone ordered — you never pay them to simply stand around waiting between orders, unlike a full-time staff member (an always-running EC2/ECS server) who's paid for their entire shift regardless of how busy it actually is.

5. Edge Cases

Case	Result
A Lambda function not invoked for a while, then suddenly called	"Cold start" — a brief extra delay while AWS provisions a fresh execution environment, before your code actually runs
A Lambda function needing to run for a VERY long time	Lambda has a MAXIMUM execution time limit (15 minutes) — genuinely long-running processes need EC2/ECS instead

6. Frequently Asked Interview Questions

- 1 **What is AWS Lambda?** A serverless compute service running your code on demand in response to triggers, automatically scaling, with billing based purely on actual execution time — no server to provision or manage at all.
- 2 **What is a Lambda "cold start"?** The brief extra latency incurred when AWS needs to provision a fresh execution environment for a function that hasn't been recently invoked.
- 3 **When would you choose Lambda over EC2/ECS?** For infrequent, event-triggered, short-duration tasks with unpredictable/spiky traffic, where you don't want to pay for continuously-running idle capacity — not for long-running, stateful, always-on applications.
- 4 **Name three common Lambda triggers.** An API Gateway HTTP request, an S3 file upload event, and a scheduled CloudWatch Event (cron-like schedule).

7. Revision Notes (Entire AWS Section Consolidated)

AWS — CHEAT SHEET

=====

CLOUD MODELS: IaaS (EC2 - you manage everything on top of the raw server) /

PaaS (Lambda-ish, Elastic Beanstalk - Amazon manages the runtime) /

SaaS (a finished app you just use)

FIRST STEPS: sign up -> NEVER use root day-to-day -> create an IAM user ->

set a billing budget alert

IAM: Users/Groups/Roles/Policies - controls WHO can do WHAT - least privilege!

EC2: virtual servers (IaaS) - AMI + instance type + key pair + security group

STOP (still pay storage) vs TERMINATE (fully removed, no more charges)

S3: object storage - globally unique bucket names, "Block Public Access" ON by default, S3 Standard/IA/Glacier for cost tiers

RDS: MANAGED relational database (Postgres/MySQL) - AWS handles patching, backups, HA - same JDBC connection code as self-hosted, just a different URL

VPC: your own isolated network - public subnet (internet-facing, e.g. Load

Balancer) vs private subnet (no direct internet access, e.g. database)

LOAD BALANCER: distributes traffic across instances, routes AROUND unhealthy ones via health checks (ties to Docker HEALTHCHECK / Spring Actuator)

AUTO SCALING: min/desired/max instance count, scaling POLICIES react to real metrics (CPU, request count) - the actual "pay for what you use" engine

CLOUDWATCH: metrics + logs + ALARMS (automated thresholds/notifications)

ROUTE 53: DNS - translates domain names to AWS resources (A/ALIAS records)

ECS: container orchestration for Docker images - EC2 launch type (you manage servers) vs FARGATE (serverless, AWS manages everything underneath)

ECR = AWS's private container registry

LAMBDA: serverless FUNCTIONS, triggered by events (HTTP/S3/schedule), pay

ONLY for actual execution time, "cold start" delay, 15-min max runtime - best for infrequent/spiky/event-driven tasks, NOT long-running stateful apps

8. Homework (Covers Entire AWS Section)

- 1 Create an AWS account, immediately set up a non-root IAM user and a billing budget alert.
- 2 Launch a `t3.micro` EC2 instance, SSH into it, install Java 17, and deploy a simple Spring Boot jar, accessing it via its public IP.
- 3 Create an S3 bucket, and modify a Spring Boot file-upload endpoint (from the Spring Boot section) to store files in S3 instead of local disk.
- 4 Create an RDS PostgreSQL instance and repoint your application's `spring.datasource.url` to it instead of your local database.
- 5 Containerize your app (Docker section), push it to ECR, and deploy it via an ECS Fargate service behind an Application Load Balancer.

FINAL SUMMARY — AWS COMPLETE

This concludes the **AWS** section — 11 topics (plus the foundational cloud models/account-setup introduction):

- **Introduction:** IaaS/PaaS/SaaS cloud service models, full account setup steps.
 1. **IAM** — identities, groups, roles, policies, least privilege.

2. **EC2** — virtual servers, full Spring Boot deployment walkthrough.
3. **S3** — object storage, replacing local-disk file uploads.
4. **RDS** — managed PostgreSQL, the same JDBC code, a different (managed) host.
5. **VPC** — public vs private subnets, the standard secure architecture pattern.
6. **Load Balancer** — traffic distribution, health-check-based routing.
7. **Auto Scaling** — automatic instance count adjustment based on real demand.
8. **CloudWatch** — metrics, logs, alarms.
9. **Route 53** — DNS, domain-to-resource resolution.
10. **ECS** — container orchestration, EC2 vs Fargate launch types.
11. **Lambda** — serverless functions, event-driven, pay-per-execution.

This is where everything built across this ENTIRE course — the Spring Boot API, the PostgreSQL database, the Docker containers, the Kafka event streams — actually gets deployed and run at real-world scale. You now understand the full path from writing Java code to serving live production traffic on the cloud.

(END OF AWS MASTER NOTES — ready to proceed to React whenever you say "Next Topic.")